

Chapter 12: Peer-to-Peer Agent Pattern, Decentralized Coordination, and Agent Collaboration Networks

Part II: Agentic Architecture Patterns

Book: Agentic AI & RAG Systems Handbook

Version: 1.0.0

Chapter ID: 12

File Name: chapter-12-peer-to-peer-agent-pattern.md

Chapter Overview

The Supervisor Pattern introduced centralized coordination:

1	Supervisor
2	↓
3	Specialists

However, some systems require a different model.

Instead of a central coordinator:

1	Agents
2	↔
3	Agents
4	↔
5	Agents

This is the **Peer-to-Peer (P2P) Agent Pattern**.

In a P2P architecture:

- Agents communicate directly
- Decisions emerge collaboratively
- Work is distributed
- Central orchestration is minimized

While supervisor architectures dominate enterprise deployments, Peer-to-Peer systems are increasingly important for:

- Research platforms
- Knowledge discovery systems
- Agent marketplaces

- Federated organizations
 - Autonomous swarms
 - Distributed intelligence systems
-

Learning Objectives

After completing this chapter you should be able to:

- Understand Peer-to-Peer architectures
 - Design decentralized agent collaboration
 - Implement agent discovery systems
 - Build federated agent networks
 - Design agent messaging protocols
 - Understand P2P governance challenges
 - Scale decentralized agent ecosystems
 - Compare P2P architectures with supervisor models
 - Explain P2P systems in architect interviews
-

Prerequisites

Required:

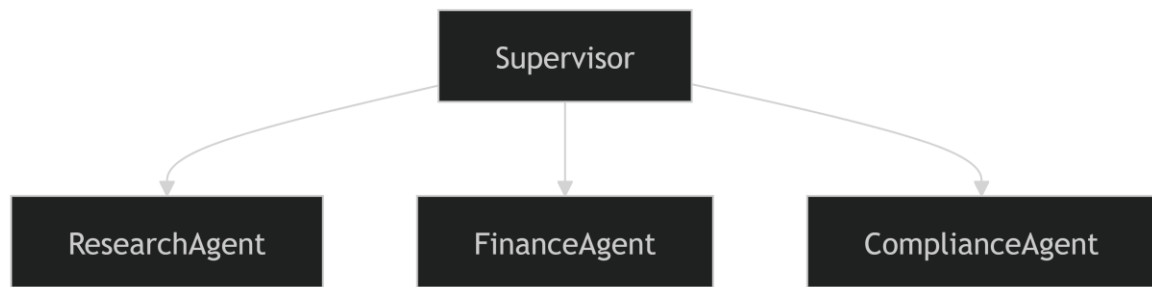
- Chapter 2: Agentic AI Fundamentals
- Chapter 4: ReAct Pattern
- Chapter 5: Router Pattern
- Chapter 11: Supervisor Architectures

Recommended:

- Future Chapters on A2A and MCP
-

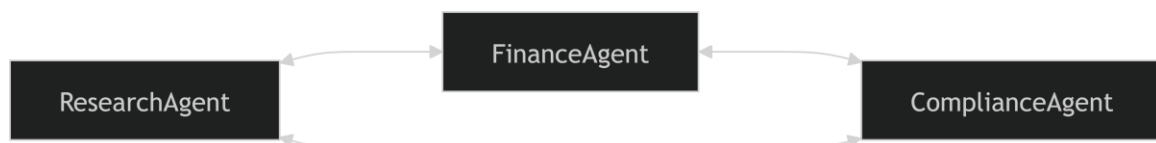
1. Introduction

Imagine a supervisor architecture:



All decisions pass through the supervisor.

Now imagine removing it.



Agents collaborate directly.

This architecture creates:

- Greater flexibility
- Greater autonomy
- Greater scalability

But also:

- Greater complexity
 - More governance challenges
 - Harder observability
-

2. Core Concepts

What Is a Peer-to-Peer Agent Architecture?

Definition

A Peer-to-Peer Agent Architecture is a distributed system where agents communicate directly with one another without requiring a central orchestrator.

Each agent may:

- Request information
- Delegate tasks

- Negotiate outcomes
 - Share knowledge
 - Coordinate actions
-

Core Principle

- | | |
|---|-----------------------|
| 1 | No Single Controller |
| 2 | |
| 3 | Shared Responsibility |

3. Why Peer-to-Peer Exists

Some environments are too dynamic for centralized orchestration.

Examples:

Research Systems

Specialized agents continuously exchange findings.

Knowledge Discovery

Relationships emerge dynamically.

Federated Organizations

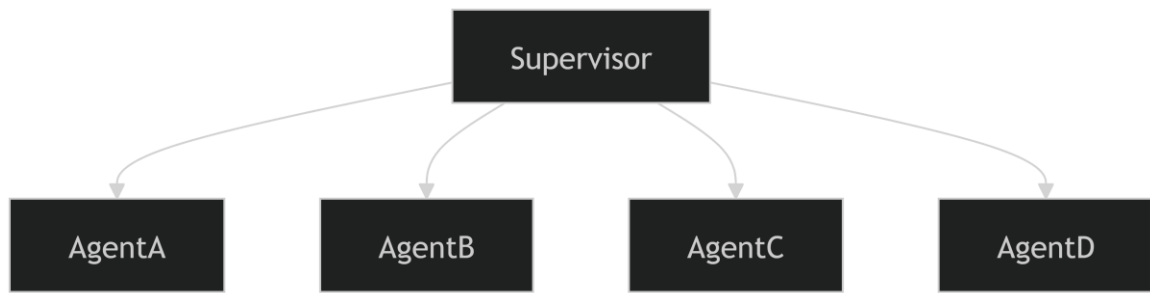
Different business units own their own agents.

Autonomous Networks

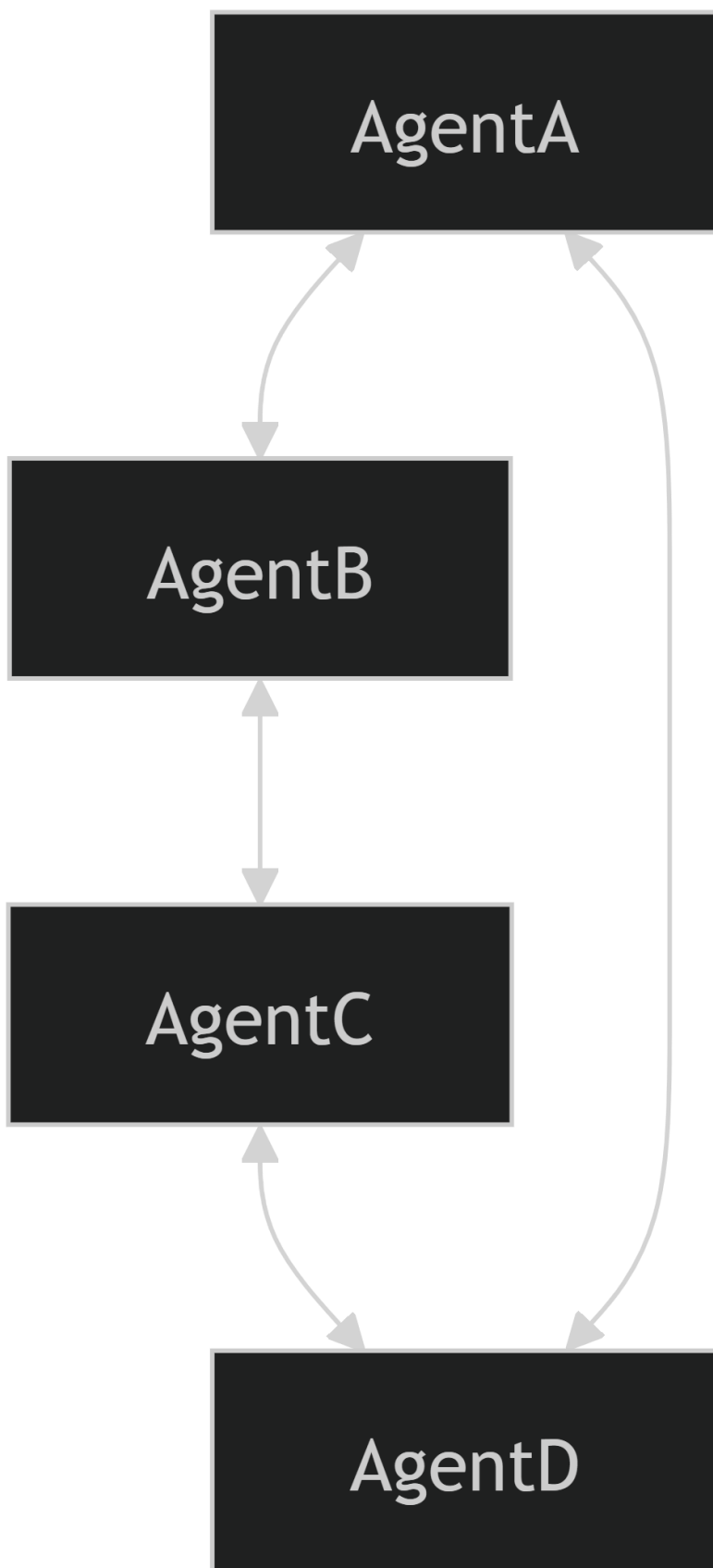
Central coordination becomes a bottleneck.

4. P2P Architecture Fundamentals

Traditional Centralized Model



P2P Model



Each participant becomes both:

- | | |
|---|----------|
| 1 | Consumer |
| 2 | and |
| 3 | Provider |

5. Human Analogy

Corporate Example:

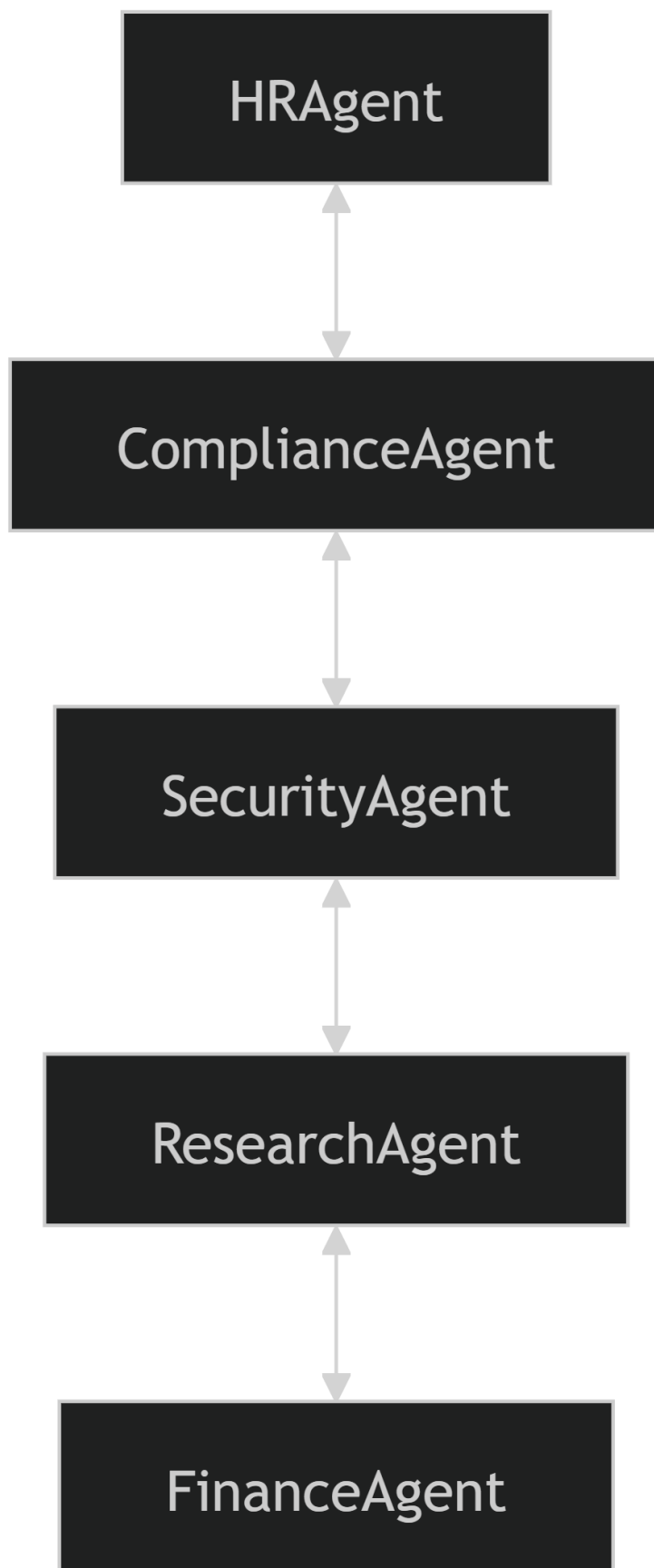
- | | |
|---|-----------------|
| 1 | Finance Team |
| 2 | ↔ |
| 3 | Legal Team |
| 4 | ↔ |
| 5 | Security Team |
| 6 | ↔ |
| 7 | Compliance Team |

No single manager mediates every interaction.

Teams collaborate directly.

P2P agents mirror this behavior.

6. Enterprise P2P Architecture



Communication occurs directly between participants.

7. Agent Discovery

One major challenge:

1 How do agents find one another?

Static Discovery

Agents configured explicitly.

Example:

<pre>1 peers: 2 - finance_agent 3 - compliance_agent</pre>
--

Advantages:

- Simple
- Predictable

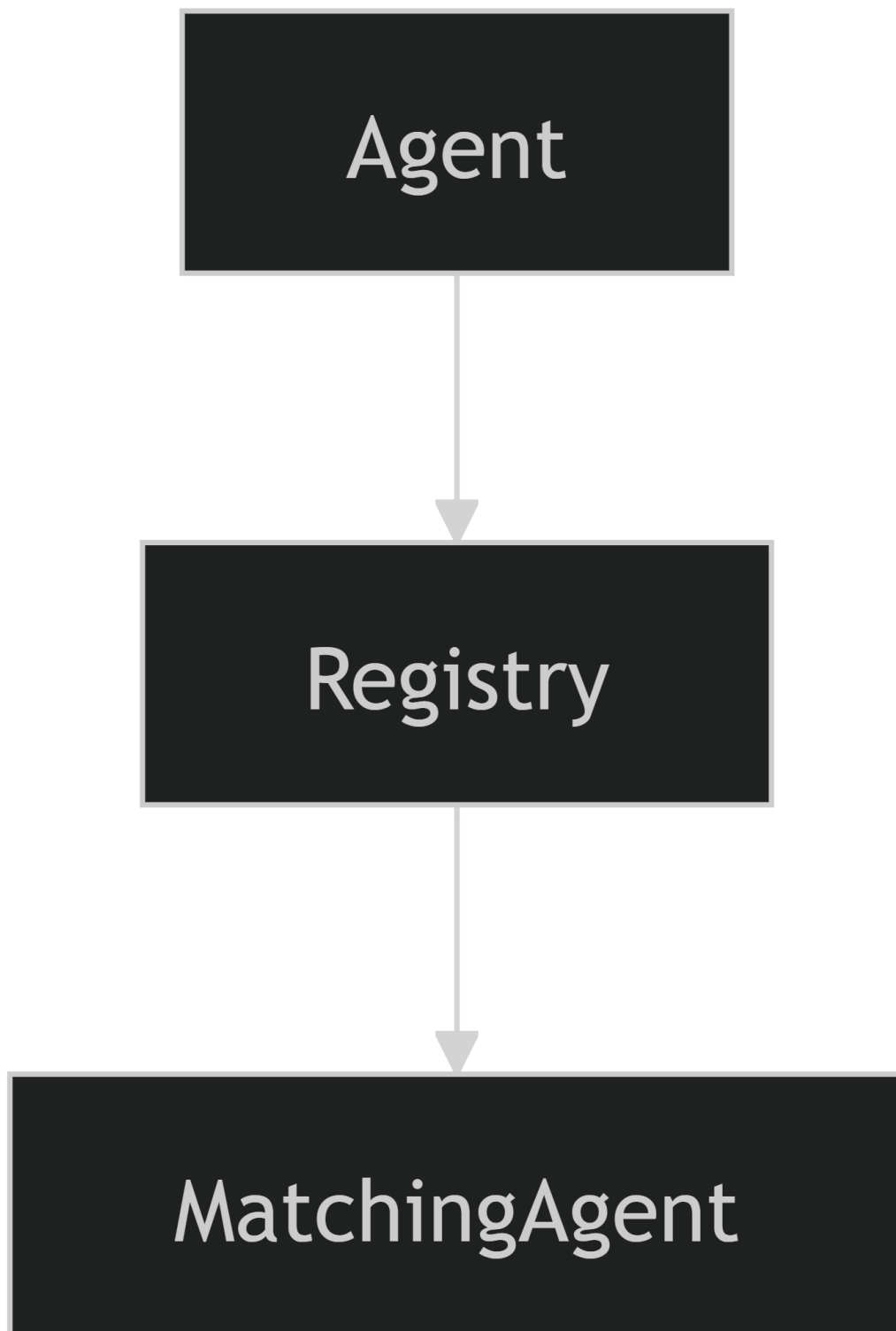
Disadvantages:

- Poor scalability
-

Dynamic Discovery

Agents locate peers at runtime.

Architecture:



Benefits:

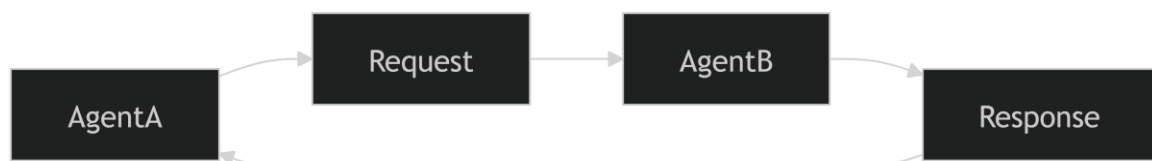
- Scalability

- Flexibility
 - Federation
-

8. Communication Models

Request–Response

Most common.



Example:

1	Research Agent
2	asks
3	Finance Agent
4	for revenue analysis.

Delegation

An agent transfers responsibility.

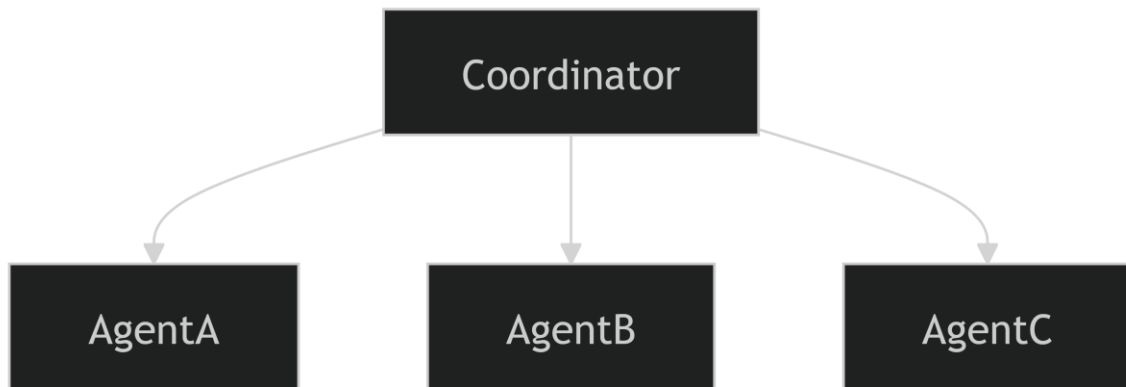
1	Agent A
2	↓
3	Agent B

Example:

1	Research Agent
2	delegates
3	industry analysis.

Broadcast

One message reaches multiple peers.

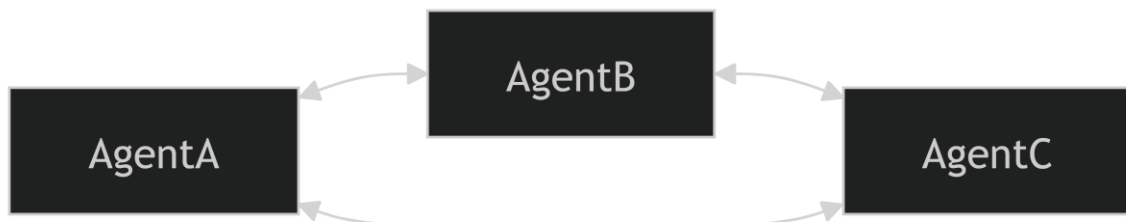


Useful for:

- Alerts
 - Notifications
 - Coordination events
-

Negotiation

Several agents collaborate toward an outcome.



Most complex communication model.

9. Agent Messaging

P2P systems require clear message contracts.

Message Structure

```
1 {  
2   "message_id": "123",
```

```
3  "sender":"ResearchAgent",
4  "receiver":"FinanceAgent",
5  "workflow_id":"ABC",
6  "task":"AnalyzeRevenue",
7  "payload":{}
8  }
```

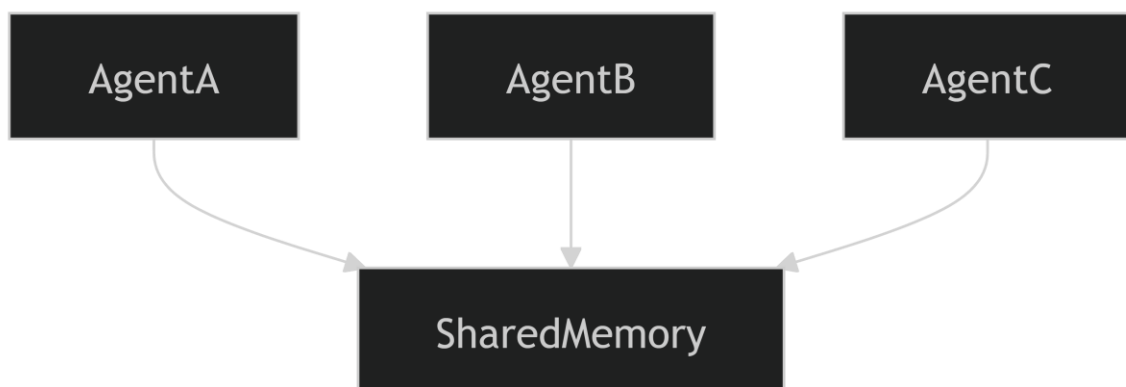
Required fields:

- Sender
 - Receiver
 - Correlation ID
 - Workflow ID
 - Timestamp
 - Context
-

10. Shared Memory Pattern

Many P2P systems use shared memory.

Architecture:



Benefits:

- Reduced messaging
 - Shared context
 - Better collaboration
-

11. Blackboard Architecture

A special form of shared memory.

1 Knowledge Board

Agents:

1 Read
2 Write
3 Update

continuously.

Useful in:

- Research systems
 - Investigation systems
 - Knowledge discovery platforms
-

12. Design Patterns

Pattern 1: Collaborative Research

Agents specialize in:

- Industry
- Finance
- Technology

Outputs merged later.

Pattern 2: Distributed Investigation

Agents independently investigate different evidence.

Example:

1 Logs
2 Metrics
3 Incidents
4 Knowledge Base

Pattern 3: Knowledge Swarm

Large number of agents collaborate.

Useful for:

- Discovery
 - Exploration
 - Research
-

Pattern 4: Federated Enterprise Network

Departments maintain independent agents.

Collaboration occurs through contracts.

13. LangGraph Peer-to-Peer

LangGraph can implement decentralized communication.

Architecture:

AgentA



AgentB



AgentC

Shared State Example

```
1 class SharedState(TypedDict):  
2     findings: list  
3     messages: list
```

14. AutoGen Peer Systems

AutoGen naturally supports agent conversations.

Example:

```
1 Research Agent  
2     ⇔  
3 Finance Agent  
4     ⇔  
5 Legal Agent
```

This is one of the strongest fits for P2P architectures.

15. CrewAI Peer Teams

CrewAI generally prefers manager-worker models, but peer collaboration can still be implemented.

Suitable for:

- Research
 - Planning
 - Knowledge exploration
-

16. Semantic Kernel Peer Patterns

Semantic Kernel supports decentralized interactions through skill orchestration and function coordination.

Typically used in enterprise integrations.

17. Agent-to-Agent (A2A)

Modern P2P systems increasingly rely on A2A standards.

Benefits:

- | | |
|---|---------------|
| 1 | Identity |
| 2 | Discovery |
| 3 | Messaging |
| 4 | Authorization |
| 5 | Governance |

A2A becomes the protocol foundation.

18. MCP in Peer Systems

MCP solves:

- | | |
|---|----------------------|
| 1 | Tool Standardization |
|---|----------------------|

P2P solves:

- | | |
|---|---------------------|
| 1 | Agent Collaboration |
|---|---------------------|

Together they create powerful ecosystems.

19. Enterprise Example: Research Platform

Agents:

- | | |
|---|-------------------|
| 1 | Market Agent |
| 2 | |
| 3 | Competitive Agent |
| 4 | |
| 5 | Technology Agent |
| 6 | |
| 7 | Financial Agent |

Each contributes evidence.

Conclusions emerge collaboratively.

20. Enterprise Example: Root Cause Analysis

Agents investigate:

```
1  Infrastructure
2
3  Applications
4
5  Databases
6
7  Deployment History
```

Insights are exchanged across peers.

Final diagnosis emerges from collaboration.

21. Security

P2P systems introduce additional risks.

Identity

Every agent must have identity.

```
1  agent_id:
2  version:
3  owner:
```

Authorization

Not every agent may communicate with every other agent.

Policy examples:

```
1  research_agent:
2    can_call:
3      - finance_agent
```

Audit Logging

Track:

- | | |
|---|-------------------|
| 1 | Who communicated? |
| 2 | |
| 3 | What was sent? |
| 4 | |
| 5 | Why? |

22. Governance

Central challenge of P2P architectures.

Without governance:

- | | |
|---|-------------|
| 1 | Agent Chaos |
|---|-------------|

Architecture:

Agent



PolicyEngine



PeerAgent

23. Observability

Track:

Messages Sent

Communication volume.

Delegations

Task transfers.

Conversation Depth

Interaction chains.

Communication Failures

Errors.

Agent Utilization

Workload distribution.

24. Scalability

Small Networks

1	10 agents
---	-----------

manageable.

Large Networks

1	1,000+ agents
---	---------------

require:

- Registries
 - Discovery protocols
 - Governance layers
 - Rate limiting
-

25. Failure Modes

Failure 1

Infinite Conversation

Agents continuously exchange messages.

Failure 2

Circular Delegation

1 A → B → C → A

Failure 3

Message Storms

Excessive communication.

Failure 4

Lost Ownership

Nobody responsible.

Failure 5

Governance Breakdown

Unauthorized collaboration.

26. Common Mistakes

Mistake 1

No identity framework.

Mistake 2

No communication contracts.

Mistake 3

No discovery mechanism.

Mistake 4

Unlimited agent interactions.

Mistake 5

No observability.

27. Anti-Patterns

Everybody Talks To Everybody

Unscalable.

Hidden Communication

Impossible to audit.

No Termination Criteria

Infinite loops.

Unbounded Networks

No governance boundaries.

28. Trade-Offs

Advantage	Cost
Flexibility	More complexity
Resilience	Harder governance
Scalability	More observability requirements
Distributed decisions	Less predictability

29. Peer-to-Peer vs Supervisor

Dimension	Supervisor	P2P
Governance	Strong	Harder
Scalability	Strong	Strong
Debugging	Easier	Harder
Flexibility	Moderate	Excellent
Observability	Easier	More Difficult
Enterprise Adoption	High	Moderate

30. Architecture Decision Matrix

Scenario	P2P Fit
Research Platform	Excellent
Knowledge Discovery	Excellent
Root Cause Analysis	Strong
Federated Enterprise	Strong
Compliance Platform	Moderate
Enterprise Copilot	Usually Supervisor Better
Financial Systems	Usually Supervisor Better

31. Real-World Use Cases

Research Ecosystems

Knowledge sharing.

Autonomous Investigation

Distributed evidence gathering.

Federated Organizations

Independent departments.

Knowledge Swarms

Large-scale discovery.

Agent Marketplaces

Dynamic capability collaboration.

32. Interview Questions

Q1

What is a P2P agent architecture?

Q2

Why use P2P systems?

Q3

What is the biggest P2P challenge?

Q4

How do agents discover one another?

Q5

What is a communication contract?

Q6

How does A2A relate to P2P?

Q7

Why are supervisor systems often preferred?

Q8

When should P2P be selected?

33. Interview Answers

Q1

A decentralized architecture where agents communicate directly without central supervision.

Q2

To improve flexibility, collaboration, and distributed decision making.

Q3

Governance and observability.

Q4

Through static configuration or capability registries.

Q5

A defined message structure and communication agreement.

Q6

A2A provides standardized communication between peer agents.

Q7

They provide stronger governance and simpler operations.

Q8

For research, discovery, federation, and collaborative intelligence systems.

34. Architecture Review Checklist

P2P Readiness

- Agent identities defined
 - Communication contracts documented
 - Discovery mechanism implemented
 - Governance controls enabled
 - Audit logging configured
 - Observability implemented
 - Termination rules defined
 - Security reviewed
 - Scaling model documented
-

35. Practical Exercises

1. Design a federated P2P agent architecture.
 2. Define an A2A message contract.
 3. Build a capability discovery registry.
 4. Compare P2P and Supervisor patterns.
 5. Design governance controls for decentralized agents.
-

36. Key Takeaways

- P2P systems eliminate centralized coordination.
 - Agents collaborate directly.
 - Discovery becomes a critical architectural concern.
 - Governance is harder than supervisor-based systems.
 - A2A standards are highly relevant.
 - Shared memory can simplify collaboration.
 - Message contracts improve reliability.
 - P2P architectures excel in research and discovery workloads.
 - Observability is significantly more important in decentralized systems.
 - Supervisor and P2P patterns should be selected based on operational requirements, not ideology.
-

37. References

- Distributed Multi-Agent Systems Research
 - AutoGen Documentation
 - LangGraph Documentation
 - CrewAI Documentation
 - Semantic Kernel Documentation
 - Agent-to-Agent (A2A) Protocol Specifications
 - Federated Agent Architecture Patterns
-

Chapter Status: Completed

Markdown Ready: Yes

PDF Ready: Yes (pending build pipeline)

Next Chapter: Chapter 13 – Parallel Agent Pattern, Fan-Out/Fan-In Architectures, and High-Throughput Agent Execution Systems